The Frequency Domain

Week 6 — Seeing images as sums of waves

Practical Image Processing

Lecture 6 ▪ 60–70 min

Recap & today

- Wk3–5: filters in the **spatial** domain (pixels & kernels).
- Today: the **frequency** domain — every image is a sum of waves. Some operations are far easier here, and **periodic noise** becomes trivial to remove.

Why it matters

A striped/patterned scan of a meter is nearly impossible to clean spatially — but it's a few dots in the frequency domain.

Roadmap

The Fourier idea

Filtering in frequency

Periodic noise

In-class exercise

Deep dive

Wrap-up

The Fourier idea

Any signal = a sum of sine waves

- **Fourier**: any image can be written as a sum of 2D sinusoids of different **frequency**, **orientation**, and **amplitude**.
- Low frequencies = smooth, slow variation (overall brightness, big shapes).
- High frequencies = fast variation (edges, fine texture, noise).

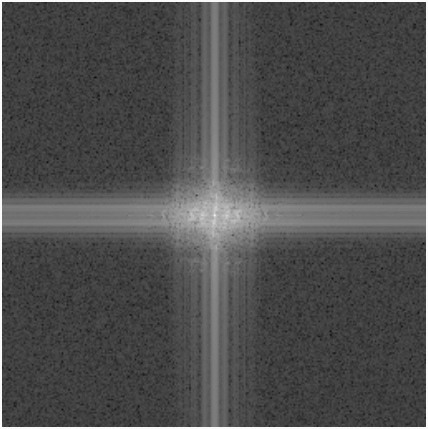
Math

2D DFT: $F(u,v) = \sum_x \sum_y f(x,y)\, e^{-j2\pi(ux/W + vy/H)}$

The spectrum of a real plate



image (spatial)



magnitude spectrum (log)

- Centre = low frequencies (DC = average brightness). Edges = high frequencies.
- Bright lines/points reveal dominant orientations and repeating structure.

Reading the spectrum

- The **centre** is the average; energy usually concentrates there (natural images are mostly low-freq).
- A strong **direction** in the image $\rightarrow$ a bright line *perpendicular* to it in the spectrum.
- **Repeating patterns** (stripes, halftone) $\rightarrow$ isolated bright **peaks** off-centre.

Note

We take the `log` of the magnitude to see anything — the DC term dwarfs everything else.

Filtering in frequency

Low-pass & high-pass, done in frequency



original



low-pass = blur



high-pass = edges

Keep the centre (low freq) → smoothing. Remove the centre (keep high) → edges. Same effects as Week 3, achieved by *masking* the spectrum.

Convolution theorem: why this matters

Math

Convolution in space = multiplication in frequency: $f * h \longleftrightarrow F \cdot H$

- A big spatial convolution becomes a cheap element-wise multiply in frequency.
- For large kernels, FFT-based filtering is much faster ($O(N \log N)$ vs $O(Nk^2)$).
- It's also *why* the spatial and frequency views are two sides of one coin.

Ideal vs smooth filters (ringing)

- An **ideal** sharp cutoff (a hard circle) causes **ringing** — ripples near edges.
- Smooth cutoffs (**Butterworth**, **Gaussian**) avoid ringing — the usual choice.
- Trade-off: sharper frequency selectivity $\leftrightarrow$ more spatial ringing.

✕ Pitfall

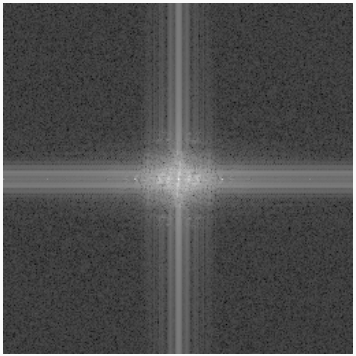
A hard frequency mask can add wavy artifacts to a clean image. Prefer a gradual roll-off.

Periodic noise

Periodic noise: the killer app



striped noise



spectrum: 2 bright peaks



after notch: clean

Key idea

Repeating stripes are a nightmare in the spatial domain — but two dots in frequency. Zero the dots (a **notch** filter), invert, done.

Notch filtering

- A **notch** filter zeroes small regions of the spectrum (the noise peaks) and keeps the rest.
- Find the peaks (they're symmetric about the centre), mask them, inverse-transform.
- Perfect for scan lines, halftone patterns, sensor interference — real document artifacts.

✓ Good practice

For DocScan: if a scanned form has a repeating background pattern, a notch filter removes it cleanly.

Fourier in code

```
F = np.fft.fftshift(np.fft.fft2(gray))      # to frequency, centre the DC
mag = 20*np.log(np.abs(F) + 1)              # log magnitude for display
# low-pass: keep a central disc
mask = make_circle_mask(gray.shape, r=30)
out = np.abs(np.fft.ifft2(np.fft.ifftshift(F * mask)))  # back to spatial
```

`fft2` → mask → `ifft2`. The whole frequency workflow in three steps.

In-class exercise

In-class exercise (10 min)

Predict the spectrum

For each image, what does its magnitude spectrum look like?

1. A perfectly flat gray image.
2. Vertical black-and-white stripes.
3. A photo with lots of fine texture vs a smooth gradient.
4. Where would you look in the spectrum to detect a repeating scan pattern?

Debrief

- (1) Flat $\rightarrow$ a single point at the centre (only DC; no variation).
- (2) Vertical stripes $\rightarrow$ two symmetric points on the horizontal axis (one frequency).
- (3) Fine texture $\rightarrow$ lots of energy far from centre; smooth $\rightarrow$ energy hugging the centre.
- (4) Look for isolated bright peaks *off-centre* — that's periodic structure.

Key idea

The spectrum is a map of "how much of each wave." Reading it becomes intuition with practice.

Deep dive

Amplitude vs phase

- The DFT has **magnitude** (how much of each frequency) and **phase** (where it sits).
- Surprising fact: **phase** carries most of the *structure* — swap two images' phases and you swap what you see.
- We visualize magnitude, but don't forget phase exists and matters.

FFT: why it's fast

- A naive DFT is $O(N^2)$; the **Fast Fourier Transform** is $O(N \log N)$.
- This is what makes frequency-domain filtering practical on real images.
- Numpy/OpenCV give you the FFT; you just prepare the mask and invert.

Math

For a 1024^2 image, FFT is $\sim$50,000$\times$ fewer operations than the naive sum.

Other transforms (brief)

- **DCT** (cosine transform): the heart of JPEG — compress by keeping low-frequency coefficients.
- **Wavelets**: multi-scale, localized in *both* space and frequency — good for denoising & compression.
- These are cousins of Fourier, each trading off localization vs frequency resolution.

When to use frequency vs spatial

- **Frequency**: periodic noise, very large filters, analyzing texture/orientation.
- **Spatial**: local operations, edges, morphology, anything position-dependent.
- Most day-to-day work is spatial; frequency is the right tool for specific jobs.

Key idea

Don't reach for the FFT by default — but when you see a repeating pattern, it's exactly the tool.

Aliasing, in the frequency view

- Week 1's aliasing is clearest here: sampling replicates the spectrum; if it's too wide, copies **overlap**.
- Overlapping spectra = high frequencies folding into low ones = the moiré/jaggies you saw.
- Blur (low-pass) before downsampling narrows the spectrum so copies don't overlap — anti-aliasing.

Homomorphic filtering (brief)

- Uneven lighting is low-frequency; reflectance (the content) is higher-frequency.
- Take the log, then high-pass slightly to suppress illumination and boost detail.
- Useful for a document photographed under a gradient of light — flattens the shading.

✓ Good practice

Another DocScan trick: homomorphic filtering can even out lighting before thresholding.

Frequency thinking $\rightarrow$ deep learning

- CNNs learn spatial kernels, but the frequency view explains what they capture (edges = high-freq).
- Some modern architectures operate partly in frequency for efficiency.
- Understanding frequency deepens your intuition for what any filter — hand-made or learned — does.

Wrap-up

What you should now be able to do

- Explain an image as a sum of waves; read a magnitude spectrum.
- Low/high-pass filter by masking the spectrum; know about ringing.
- Remove periodic noise with a notch filter (the killer application).
- Use the convolution theorem to reason about speed and equivalence.

This week's deliverable

Due before Week 7 (see `project/`)

1. Compute and display the magnitude spectrum of 3 real images; interpret each.
2. Implement low-pass and high-pass by masking; compare to spatial blur/edges.
3. Add periodic (stripe) noise; locate the peaks; remove it with a notch filter.
4. Note: when is frequency-domain the right tool for a DocScan problem, and when not?

An image is a sum of waves.

Some problems that are impossible in space are trivial in frequency.